

Chương 18: Học Tăng Cường (Reinforcement Learning)

Giảng viên

Đại học Đông Á

August 11, 2026

Mục tiêu của Chương

- Nắm vững khái niệm cơ bản về Học Tăng Cường: **Tác nhân, Môi trường, Phần thưởng.**
- Hiểu nguyên lý và cách tìm kiếm chính sách (Policy Search).
- Phân tích và áp dụng thuật toán **Học Q (Q-Learning)** thông qua bảng.
- Khám phá sức mạnh của **Deep Q-Learning (DQN)** và các kỹ thuật giúp nó hội tụ.
- Nắm bắt cơ sở lý thuyết toán học của **Markov Decision Processes (MDP)**.
- Triển khai mạng nơ-ron học **Độ dốc chính sách (Policy Gradients)**.

Mục lục

1. Học để tối ưu hóa phần thưởng & Tìm kiếm chính sách
2. Markov Decision Processes (MDP)
3. Học Q (Q-Learning) Cổ điển
4. Học Q Sâu (Deep Q-Learning)
5. Độ dốc Chính sách (Policy Gradients)
6. Actor-Critic và Các Kiến trúc Hiện đại
7. Tổng kết

1.1 Bối cảnh của Học Tăng Cường

- Trong Học Máy, chúng ta đã quen với **Học có giám sát** (có nhãn sẵn) và **Học không giám sát** (tìm quy luật).
- **Học Tăng Cường (Reinforcement Learning - RL)** là một mô hình hoàn toàn khác.
- Nó xoay quanh một hệ thống đang tự động học hỏi bằng cách *tương tác* với môi trường xung quanh.
- Mục tiêu duy nhất: Tối đa hóa tổng **Phần thưởng (Reward)** nhận được theo thời gian.

1.2 Mô hình Tác nhân và Môi trường

- **Tác nhân (Agent):** Chủ thể quan sát và đưa ra quyết định (VD: Robot, phần mềm chơi cờ).
- **Môi trường (Environment):** Thế giới vật lý hoặc giả lập mà Tác nhân thao tác.
- Môi trường cung cấp **Trạng thái** và **Phần thưởng**. Tác nhân tác động lại bằng **Hành động**.



Hình 18-1:

Chu trình Học Tăng Cường cơ bản

1.3 Ví dụ thực tế về RL

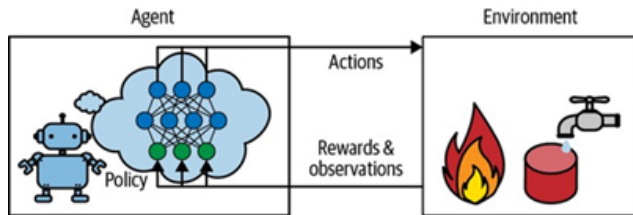
- **Điều khiển Robot:** Robot học cách đi bộ. Nếu đi được, +1 điểm. Nếu ngã, -10 điểm. Qua hàng triệu lần ngã, nó tự biết cách thăng bằng.
- **Chơi Game:** AlphaGo của DeepMind đánh bại nhà vô địch thế giới cờ vây Lee Sedol. Phần thưởng chỉ được trao ở cuối ván (Thắng = +1, Thua = -1).
- **Quản lý danh mục đầu tư:** Tác nhân mua/bán cổ phiếu dựa trên trạng thái thị trường. Phần thưởng là lợi nhuận thu về.

1.4 Chính sách (Policy)

- **Chính sách (ký hiệu là π):** Là thuật toán hoặc hàm số mà Agent dùng để ra quyết định.
- Phân loại chính sách:
 - **Chính sách xác định (Deterministic Policy):** Ở trạng thái S , luôn chọn hành động A (VD: $A = \pi(S)$).
 - **Chính sách ngẫu nhiên (Stochastic Policy):** Ở trạng thái S , có 80% chọn A , 20% chọn B .
- Nhiệm vụ cốt lõi của RL là đi tìm một chính sách tối ưu π^* .

1.5 Tìm kiếm chính sách (Policy Search)

- Không gian của tất cả các chính sách có thể có là vô cùng tận. Làm sao để tìm ra chính sách tốt?
- **Cách tiếp cận:**
 1. *Tìm kiếm ngẫu nhiên*: Đặt tham số mạng nơ-ron ngẫu nhiên, cho chạy thử. Rất kém hiệu quả.
 2. *Thuật toán di truyền*: Lai ghép các chính sách tốt.
 3. *Gradient Ascent*: Dùng đạo hàm để leo lên đỉnh núi phần thưởng.



Hình 18-2: Không gian chính sách và quá trình tìm đỉnh

1.6 Môi trường OpenAI Gym (Gymnasium)

- Để huấn luyện AI, ta cần một môi trường giả lập chuẩn hóa.
- **OpenAI Gym** ra đời để cung cấp một API thống nhất cho hàng loạt các môi trường khác nhau.
- Các bước tương tác cơ bản với Gym:
 - `env.reset()`: Khởi động lại môi trường.
 - `env.step(action)`: Yêu cầu môi trường thực thi hành động. Trả về: (state, reward, done, info).

1.7 Cấu trúc Code cơ bản với OpenAI Gym

```
import gym

env = gym.make("CartPole-v1")
obs = env.reset()

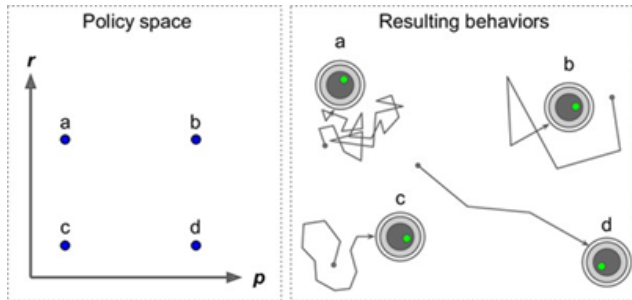
for step in range(1000):
    env.render()    # Hien thi hinh anh
    action = env.action_space.sample() # Chon hanh dong ngau nhien
    obs, reward, done, info = env.step(action)

    if done:
        print(f"Game over sau {step} buoc!")
        break
env.close()
```

- Code minh họa một Agent ngẫu nhiên chọn hành động hoàn toàn ngẫu nhiên.

1.8 Bài toán CartPole

- **Nhiệm vụ:** Giữ gậy đứng thẳng trên chiếc xe.
- **Không gian hành động:**
 - 0: Đẩy sang trái
 - 1: Đẩy sang phải
- **Không gian quan sát (Observation):** [Vị trí xe, Vận tốc xe, Góc nghiêng gậy, Vận tốc góc].
- **Done (Kết thúc):** Khi gậy nghiêng quá 15° hoặc xe chạy ra khỏi màn hình.



Hình 18-3: Giao diện giả lập CartPole

1.9 Các Chính sách bằng Mạng Nơ-ron

- Thay vì dùng If-Else, ta dùng một **Mạng Nơ-ron MLP** (Multi-Layer Perceptron) để làm Chính sách.
- Đầu vào: Một mảng các quan sát (Ví dụ: 4 giá trị của CartPole).
- Lớp ẩn: Xử lý thông tin phi tuyến.
- Đầu ra: Xác suất chọn các hành động (Dùng hàm Softmax cho đa lớp, hoặc Sigmoid cho nhị phân).
- Trọng số của mạng sẽ được cập nhật liên tục qua quá trình huấn luyện.

1.10 Bài toán Phân bổ tín nhiệm (Credit Assignment)

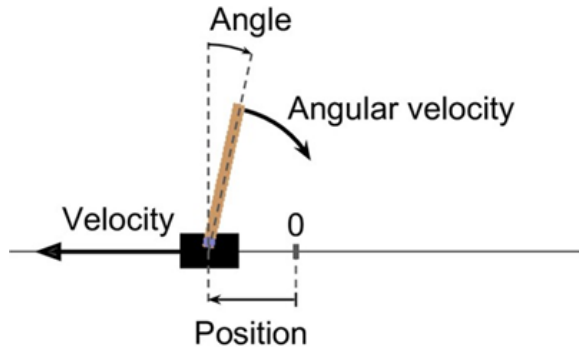
- Khi đánh cờ, bạn thực hiện 100 nước đi. Đến nước thứ 101, bạn chiến thắng và nhận thưởng +100.
- Cả 100 nước đi đó đều là nước tốt? Chưa chắc, có thể nước 50 rất tệ nhưng nước 99 cực kỳ xuất sắc.
- **Làm thế nào để hệ thống biết chính xác hành động nào trong quá khứ xứng đáng được biểu dương (phân bổ tín nhiệm)?**
- Giải pháp phổ biến: Sử dụng hệ số chiết khấu và thuật toán *Truy ngược phần thưởng* (*Reward Backpropagation*).

2.1 Nền tảng Toán học: Quá trình Markov

- **Quá trình Markov (Markov Chain):** Một hệ thống chuyển đổi giữa các trạng thái theo xác suất cố định, không có bộ nhớ (Memoryless).
- "Tương lai chỉ phụ thuộc vào trạng thái hiện tại, không liên quan đến quá khứ".
- Trạng thái thời tiết: Từ Nắng $\rightarrow$ có 70% sang Mưa, 30% tiếp tục Nắng.

2.2 Quá trình Quyết định Markov (MDP)

- MDP là bản nâng cấp của Markov Chain, bổ sung thêm 2 thành phần cốt lõi:
 - **Hành động (Action):** Ta có quyền can thiệp vào quá trình chuyển trạng thái.
 - **Phần thưởng (Reward):** Nhận được sau khi chuyển trạng thái.
- Đây là cách biểu diễn toán học chuẩn xác nhất cho mọi bài toán RL.



Hình 18-4: Sơ đồ minh họa MDP với Trạng thái, Hành động và Xác suất

2.3 Hệ số chiết khấu (Discount Factor - γ)

- Agent không chỉ tìm kiếm phần thưởng lập tức, mà phải tối ưu phần thưởng dài hạn.
- Tuy nhiên, 10 điểm ngay bây giờ luôn chắc chắn hơn 10 điểm vào tuần sau.
- Ta định nghĩa **Lợi nhuận (Return G_t)** tại bước t :

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

- γ (gamma) thường lấy giá trị từ 0.9 đến 0.99. Nếu $\gamma = 0$, Agent rất thiên cận (chỉ nhìn trước mắt). Nếu $\gamma \rightarrow 1$, Agent nhìn xa trông rộng.

2.4 Đánh giá Trạng thái và Hành động

- Làm sao để biết một trạng thái S là tốt hay xấu?
- **Hàm giá trị trạng thái (State-Value Function - $V^\pi(s)$):** Tính tổng lợi nhuận dự kiến nhận được nếu bắt đầu từ s và tuân theo chính sách π .
- **Hàm giá trị hành động (Action-Value Function - $Q^\pi(s, a)$):** Đứng ở s , thực hiện hành động a , sau đó tuân theo chính sách π , thì thu được bao nhiêu?

2.5 Phương trình Bellman (Bellman Equation)

- Richard Bellman đã chứng minh được mối liên hệ đệ quy của các hàm Giá trị:
- Để tính $V(s)$ tại trạng thái hiện tại, ta chỉ cần phần thưởng ngay lập tức và $V(s')$ của trạng thái tiếp theo:

$$V^*(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^*(s') \right]$$

- Công thức này cho phép ta giải bài toán bằng **Quy hoạch động (Dynamic Programming)** hoặc vòng lặp.

3.1 Nguyên lý Học Q (Q-Learning)

- Q-Learning (đề xuất năm 1989 bởi Watkins) không cần biết trước môi trường (Model-free).
- Thuật toán lưu trữ tất cả các giá trị $Q(s, a)$ vào một cái Bảng khổng lồ (Q-Table).
- Ban đầu Q-Table chứa toàn số 0. Agent bắt đầu đi lung tung.
- Mỗi lần bước đi, nó cập nhật Q-Table dựa trên **Chênh lệch Thời gian (Temporal Difference - TD)**.

3.2 Công thức Cập nhật Q-Learning

- Tại mỗi bước, khi đi từ $S \rightarrow S'$ với hành động A , nhận phần thưởng R :

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[R + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

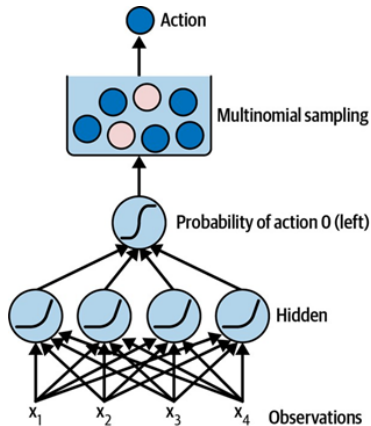
- Trong đó:
 - α (Alpha): Tốc độ học (Learning Rate).
 - $R + \gamma \max Q(s', a')$: **Mục tiêu TD (TD Target)** (Giá trị thực tế vừa nhận).
 - $[\dots]$: **Sai số TD (TD Error)** (Dự đoán sai bao nhiêu).
- Agent học bằng cách liên tục thu hẹp sai số TD.

3.3 Vấn đề: Khám phá hay Khai thác (Exploration vs Exploitation)

- Nếu Agent luôn chọn hành động có Q-value lớn nhất (Khai thác), nó sẽ mắc kẹt.
- Ví dụ: Nó tìm thấy một con đường đi an toàn được +5 điểm. Nó cứ đi mãi đường đó. Trong khi đường khác tuy mạo hiểm nhưng được +100 điểm, nó lại không bao giờ khám phá ra.
- Do đó, ta phải **ép** hệ thống có một tỷ lệ Khám phá ngẫu nhiên (Exploration).

3.4 Chính sách Epsilon-Greedy

- **Epsilon-Greedy:**
- Đặt một biến $\epsilon = 1.0$ (100% ngẫu nhiên).
- Sinh ra một số ngẫu nhiên từ $0 - 1$.
- Nếu số đó $< \epsilon$: Chọn hành động ngẫu nhiên.
- Nếu $> \epsilon$: Chọn hành động tốt nhất trong Q-Table (Khai thác).
- ϵ sẽ giảm dần theo thời gian (ví dụ giảm xuống 0.05).



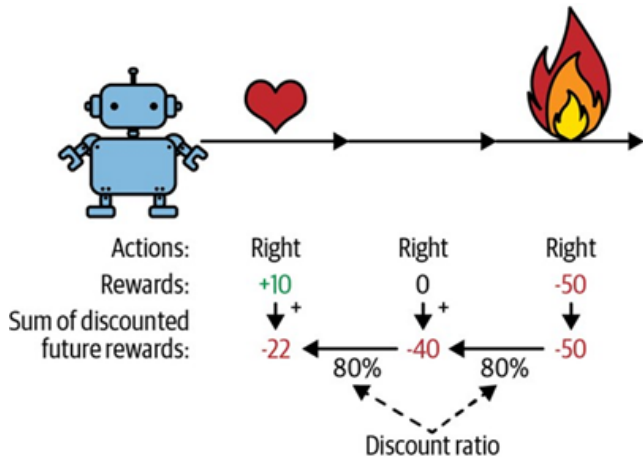
Hình 18-5: Chiến lược giảm dần tỷ lệ Epsilon theo thời gian

4.1 Khi Không Gian Trạng Thái Quá Lớn

- Q-Table cực kỳ tốn RAM. Nếu trò chơi Pacman lấy hình ảnh làm trạng thái (VD: ảnh 210×160 pixel với 256 màu), số lượng trạng thái là $256^{210 \times 160}$.
- Máy tính không thể tạo mảng 2 chiều có kích thước này.
- Giải pháp mang tính đột phá của DeepMind (2013): **Học Q Xấp xỉ (Approximate Q-Learning)**. Dùng Mạng Nơ-ron (DQN) để đoán Q-value.

4.2 Kiến trúc Deep Q-Network (DQN)

- Mạng DQN nhận **Đầu vào** là Hình ảnh màn hình Game.
- Đi qua các lớp **Convolutional (CNN)** để trích xuất đặc trưng.
- **Đầu ra** là một lớp Dense với số nơ-ron bằng chính số Hành động (Trái, Phải, Nhảy, Bắn). Mỗi nơ-ron xuất ra một giá trị Q.



Hình 18-6: Kiến trúc Mạng Nơ-ron Tích chập DQN
chơi Atari

4.3 Vấn đề Hội tụ của DQN

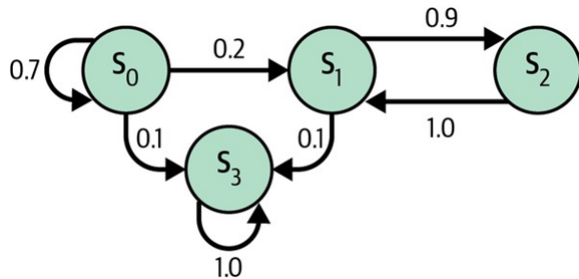
- Khác với Q-Table, dùng Mạng Nơ-ron cho RL cực kỳ thiếu ổn định (Instability) và rất hay phá vỡ những gì đã học.
- Hai lý do chính:
 - **Mẫu học có tính tương quan mạnh:** (Khung hình 1 rất giống khung hình 2). Nếu cập nhật mạng bằng dữ liệu liên tiếp, mạng sẽ "quên" cách xử lý các trường hợp xa xôi.
 - **Mục tiêu di động:** Chúng ta đang dùng chính dự đoán của mạng để làm nhãn cho mạng học, dẫn đến vòng luẩn quẩn.

4.4 Giải pháp 1: Trải nghiệm Phát lại (Replay Buffer)

- Agent không học ngay sau mỗi khung hình.
- Nó chạy quanh trò chơi và lưu tất cả kinh nghiệm (State, Action, Reward, Next State) vào một bộ nhớ khổng lồ (Replay Memory / Buffer, sức chứa khoảng 1 triệu mẫu).
- Khi huấn luyện, mạng bốc ngẫu nhiên 32 mẫu từ bộ nhớ này (Mini-batch).
- Điều này phá vỡ tính tương quan thời gian và sử dụng hiệu quả kinh nghiệm (1 mẫu được học đi học lại).

4.5 Giải pháp 2: Mạng Mục tiêu (Target Network)

- Ta tạo ra 2 mạng Nơ-ron có kiến trúc y hệt nhau:
 - **Online Network:** Mạng chính liên tục học và cập nhật.
 - **Target Network:** Dùng để tính điểm mục tiêu (Q Target). Nó bị "đóng băng" (không cập nhật).
- Cứ sau 10,000 bước, ta copy trọng số từ Mạng Online sang Mạng Target một lần.



Hình 18-7: Quá trình đồng bộ chậm từ Online sang Target

4.6 Pseudo-code cho DQN với Keras

```
# Target Network tính toán gia tri tuong lai
next_Q_values = target_model.predict(next_states)
max_next_Q_values = np.max(next_Q_values, axis=1)

# Tính nhãn (Target) bang phuong trinh Bellman
target_Q_values = rewards + (1 - dones) * gamma * max_next_Q_values

# Mask de chi chon Q-value cua hanh dong da thuc hien
mask = tf.one_hot(actions, num_actions)

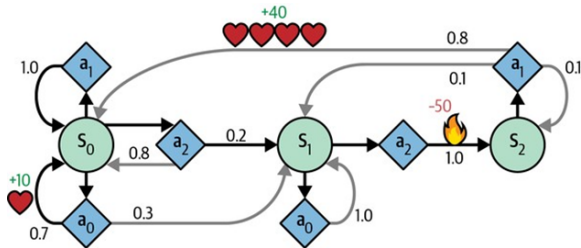
with tf.GradientTape() as tape:
    all_Q_values = model(states)
    Q_values = tf.reduce_sum(all_Q_values * mask, axis=1)
    # Tinh sai so MSE
    loss = tf.reduce_mean(keras.losses.mean_squared_error(target_Q_values, Q_values))
```

4.7 Cải tiến của DQN: Double DQN

- Mạng DQN truyền thống có xu hướng **đánh giá quá cao (Overestimate)** giá trị thật sự.
- Khi lấy $\max Q$ để làm mục tiêu, nó vô tình chọn những nơ-ron đang bị "ảo tưởng" (Nhiều tích cực).
- **Double DQN:**
 - Dùng **Mạng Online** để quyết định Đây là hành động tốt nhất.
 - Dùng **Mạng Target** để tính Giá trị thực sự của hành động đó.
 - Tách bạch người đánh giá và người chọn hành động!

4.8 Cải tiến của DQN: Dueling DQN

- Tách mạng làm 2 luồng song song sau phần tích chập CNN:
- Luồng 1 tính Giá trị Trạng thái $V(s)$.
- Luồng 2 tính Lợi thế Hành động $A(s, a)$ (Hành động này tốt hơn mức trung bình bao nhiêu).
- Hàm kết hợp: $Q(s, a) = V(s) + A(s, a)$.



Hình 18-8: Kiến trúc mạng DQN thông thường (trên) và Dueling DQN (dưới)

4.9 Ưu tiên Trải nghiệm (Prioritized Experience Replay)

- Khi bốc mẫu từ Replay Buffer, thay vì bốc ngẫu nhiên (Uniform), ta bốc theo **mức độ ưu tiên**.
- Những mẫu nào mạng dự đoán sai nhiều nhất (TD Error lớn nhất) sẽ được "chăm sóc" đặc biệt và học lại nhiều lần hơn.
- Giống như con người: Ta thường khắc sâu những sai lầm thảm khốc thay vì những công việc lặp đi lặp lại nhàm chán.

5.1 Hạn chế của Học Q với Hành động Liên tục

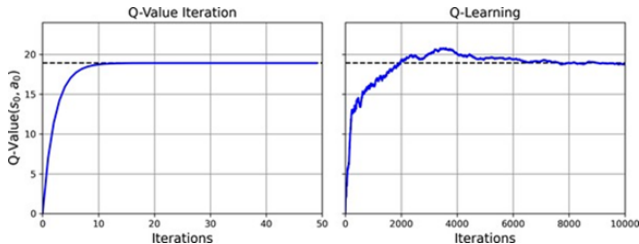
- Các phương pháp Q-Learning tính Q cho TỪNG hành động một.
- Nếu hệ thống có không gian hành động liên tục (Ví dụ: Góc xoay vô-lăng từ -90° đến $+90^\circ$ có vô số giá trị).
- Làm sao có thể lấy $\max_a Q(s, a)$ với vô hạn hành động? Q-Learning sẽ sụp đổ.
- Ta cần đổi sang các thuật toán **Policy-Based (Dựa trên Chính sách)**.

5.2 Nguyên lý Độ dốc Chính sách (Policy Gradients - PG)

- Không thêm tính giá trị Q nữa. Mạng nơ-ron nhận đầu vào là Trạng thái, và xuất trực tiếp **Xác suất chọn hành động** (softmax) hoặc xuất trực tiếp **Giá trị liên tục** (mean, std).
- **Ý tưởng cốt lõi của PG:**
 - Chơi 1 ván game trọn vẹn đến cuối.
 - Nếu kết quả chung cuộc Tốt (Phần thưởng cao), tăng xác suất của TOÀN BỘ các quyết định đã chọn trong ván đó.
 - Nếu Tệ, giảm xác suất của chúng.

5.3 Thuật toán REINFORCE (1992)

- Thuật toán kinh điển nhất của PG.
- G_t là lợi nhuận tích lũy từ bước t đến cuối ván.
- Nếu G_t là dương, Gradient sẽ kéo xác suất hành động cao lên. Nếu G_t âm, nó hạ thấp xuống.
- Mạng học trực tiếp bằng phương pháp Gradient Ascent (Leo dốc).



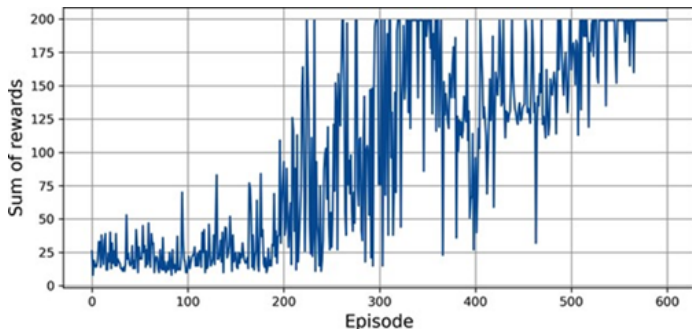
Hình 18-9: Quá trình leo dốc tối ưu hàm mục tiêu $J(\theta)$

5.4 Vấn đề Phương sai (High Variance) của REINFORCE

- Vấn đề lớn nhất của REINFORCE là phương sai cực kỳ lớn.
- Cùng một chuỗi hành động ngẫu nhiên, có khi gặp may mắn được 100 điểm, có khi gặp xui xẻo được -10 điểm. Việc phạt/thưởng mù quáng khiến quá trình huấn luyện mất ổn định.
- Giải pháp: Sử dụng **Đường Cơ sở (Baseline)**. Thay vì nhìn vào G_t , ta nhìn vào $(G_t - b)$. Tức là "Hành động này có mang lại lợi nhuận cao hơn MỨC TRUNG BÌNH không?".

6.1 Khái niệm Actor-Critic (Diễn viên & Nhà Phê Bình)

- Kết hợp đỉnh cao của 2 thể giới:
- **Actor (PG)**: Mạng chính sách quyết định hành động.
- **Critic (Q-Learning)**: Mạng giá trị đánh giá hành động đó được mấy điểm.
- Actor đưa ra quyết định $\rightarrow$ Critic chấm điểm $\rightarrow$ Actor dùng điểm đó làm Baseline để tự sửa đổi.



Hình 18-10: Cấu trúc thuật toán Actor-Critic vòng lặp khép kín

6.2 Ưu điểm của Actor-Critic

- Giải quyết bài toán phương sai cao của REINFORCE (nhờ Critic).
- Xử lý được cả không gian trạng thái liên tục và rời rạc.
- Hội tụ cực kỳ nhanh so với DQN thuần túy.
- Hầu hết các thành tựu AI lớn nhất thập kỷ qua (AlphaStar chơi Starcraft 2, OpenAI Five chơi Dota 2) đều dùng kiến trúc bắt nguồn từ Actor-Critic.

6.3 Thuật toán A3C (Asynchronous Advantage Actor-Critic)

- (DeepMind - 2016): Thay vì chỉ 1 Agent chơi game, ta thả 16, 32, hay 64 Agent con (Workers) chơi game ĐỒNG THỜI trên các luồng CPU khác nhau.
- Mỗi Agent tự chơi, tự tính Gradient, rồi gửi về Mạng Tổng (Global Network).
- Mạng Tổng chia sẻ lại kinh nghiệm cho các Agent con.
- Không cần Replay Buffer! Sự đa dạng từ các Agent con đã đủ để phá vỡ tính tương quan thời gian.

6.4 Thuật toán PPO (Proximal Policy Optimization)

- PPO (OpenAI - 2017) hiện là thuật toán mặc định vàng (Gold Standard) cho Hầu hết bài toán Học Tăng Cường.
- Khắc phục nhược điểm "bước quá dài" của PG (nhảy quá tay mất luôn cực đại).
- Cắt xén (Clipping) vùng thay đổi, ép trọng số mạng không được thay đổi quá 20% mỗi lần cập nhật.
- Đây chính là thuật toán dùng trong RLHF để tạo ra ChatGPT!

Tổng Kết Chương 18

- Học Tăng Cường mang lại trí thông minh thực sự linh hoạt cho Robot và Phần mềm, khác hẳn Học có giám sát.
- **Value-based (Q-Learning, DQN):** Học cách chấm điểm các trạng thái, phù hợp cho bài toán rời rạc. Cần dùng Replay Buffer & Target Network.
- **Policy-based (REINFORCE, PPO):** Tối ưu hóa trực tiếp xác suất hành động, phù hợp cho điều khiển liên tục.
- Sự hội tụ của 2 trường phái tạo ra **Actor-Critic**, xương sống của AI tiên tiến nhất ngày nay.

HỎI & ĐÁP